EXP-Bench: Can AI Conduct AI Research Experiments?

Taro Yamada

Abstract

Rigorous scientific experimentation requires designing hypotheses, implementing methodology, executing code, and interpreting results in a coherent, end-to-end workflow—a level of integration that existing AI benchmarks do not evaluate. Most benchmarks assess isolated capabilities such as code generation or question answering, leaving open the question of whether AI agents can conduct complete research experiments as performed in peer-reviewed AI publications. We introduce EXP-Bench, a benchmark of 461 tasks extracted semi-automatically from 51 NeurIPS and ICLR 2024 papers, comprising 12,737 fine-grained subtasks spanning computer vision, natural language processing, reinforcement learning, and applications. Each task provides a research question, a high-level method description, and a masked code repository; ground truth includes a structured design specification, a git-diff implementation, and an execution-validated conclusion. EXP-Bench evaluates agents along four dimensions—Design (D), Implementation (I), Execution (E), and Conclusion (C)—using an o3-mini LLM judge for D/I/C and Docker-based execution validation for E, with conjunctive metrics (e.g., requiring D, I, E, and C to all pass) that expose the true end-to-end capability of agents. Evaluating seven agent–model configurations across OpenHands and an IterativeAgent framework, we find that while agents achieve partial scores on individual dimensions (e.g., Implementation up to 35.0%), conjunctive all-pass rates collapse to at most 0.5%, revealing a stark gap between partial and complete experimental execution. EXP-Bench provides an execution-grounded testbed that surfaces where current AI agents break down in the scientific experimentation pipeline and establishes a challenging frontier for future AI research agents.

1 Introduction

The prospect of AI systems that autonomously conduct scientific research has attracted significant attention [10, 13, 1]. Recent LLM agents have demonstrated strong capabilities in code generation, reasoning, and information retrieval [5, 8, 12]. Yet a critical question remains: can current AI agents perform the complete workflow of a research experiment—formulating a design, implementing code, executing it, and drawing a correct conclusion—at the standard of peer-reviewed AI publications? Answering this requires a benchmark grounded in actual experimental work, not toy tasks or isolated sub-skills.

Existing benchmarks shed only partial light on this question. SWE-bench [5] evaluates repository-level code editing but not research design or scientific interpretation. MLAgentBench [4] and MLGym [11] focus on ML engineering tasks such as improving baselines, but do not require agents to extract and replicate complete experimental procedures from papers. PaperBench [14] targets paper replication from scratch but does not anchor evaluation in execution-validated, granular subtask decompositions. Scientific discovery benchmarks such as ScienceAgentBench [2] and SciCode [15] address data-driven discovery and research coding but not the full experimentation cycle in AI research. The result is that we lack a benchmark that simultaneously requires agents to (1) produce

Preprint. Under review.

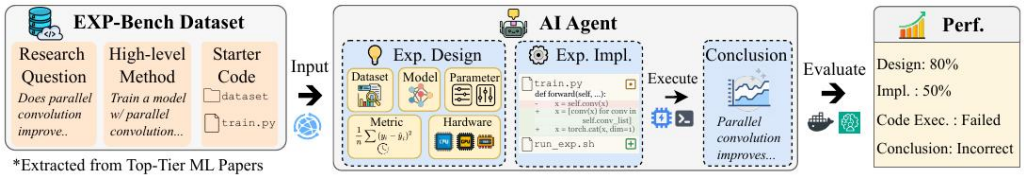

Figure 1: EXP-Bench evaluates AI agents on research experiment tasks extracted semi-autonomously from peer-reviewed AI papers. Agents are given a masked repository, a research question, and a high-level method; they must produce a design, implementation, execution, and conclusion that are evaluated jointly.

a research design grounded in a given paper, (2) implement the experiment in an existing code repository, (3) execute the code successfully, and (4) arrive at the correct scientific conclusion—with all four evaluated jointly.

We address this gap with EXP-Bench, a benchmark that evaluates AI agents on complete, executable AI research experiments extracted from peer-reviewed publications. EXP-Bench comprises 461 tasks derived from 51 NeurIPS and ICLR 2024 papers, yielding 12,737 fine-grained subtasks covering computer vision, NLP, reinforcement learning, and applications domains. Each task presents an agent with a masked version of a paper's code repository, a research question, and a high-level description of the experimental approach. The agent must produce a structured design plan, implement the experiment by modifying the repository, execute the code, and draw a conclusion aligned with the paper's findings. Ground truth for each task was constructed through a semi-automated three-stage pipeline involving multimodal PDF parsing, tool-augmented implementation extraction, and Docker-based execution validation. Integrity monitoring during agent evaluation detects disallowed shortcuts such as accessing masked paper content or generating fake results.

Our evaluation of seven agent–model configurations—using OpenHands [16] and an IterativeAgent framework with models including o3-mini, Claude-3.7 Sonnet, Claude-3.5 Haiku, DeepSeek R1, and Amazon Nova Pro—reveals that current agents achieve meaningful scores on individual metrics (e.g., the best Implementation score is 35.0% for OpenHands+Claude-3.7 Sonnet), but conjunctive all-pass rates that require Design, Implementation, Execution, and Conclusion to all succeed simultaneously collapse to at most 0.5%. A detailed failure taxonomy reveals that missing implementation components (39.7%), environment/dependency errors (29.4%), and incorrect or absent conclusions (26.2% and 19.7%) are the most prevalent failure modes. Our contributions are:

- **EXP-Bench**: the first benchmark to evaluate AI agents on complete, executable AI research experiments, grounded in peer-reviewed NeurIPS/ICLR 2024 papers.
- A **semi-automated, scalable construction pipeline** that extracts research tasks and executable implementations from papers and code repositories with lightweight human review.
- A **diverse dataset** of 461 tasks (12,737 subtasks) with execution-validated ground truths and masked repositories that prevent shortcutting.
- A **rigorous conjunctive evaluation protocol** combining LLM judging, Docker-based execution validation, and integrity monitoring, plus a detailed failure taxonomy diagnosing where agents break down.

2 Related Work

2.1 AI Agent Benchmarks

A growing body of work benchmarks LLM agents on interactive, multi-step tasks. SWE-bench [5] evaluates repository-level code editing via real GitHub issues. AgentBench [8] evaluates agents across diverse environments including operating systems, databases, and web browsers. RE-Bench [17] benchmarks AI R&D engineering with 7 open-ended ML tasks against human expert timelines. MLAgentBench [4] focuses on ML experimentation tasks such as improving model performance, and MLGym [11] provides a gym-style environment for 13 AI research tasks. While these benchmarks

capture important dimensions of agent capability, none jointly evaluate designing, implementing, executing, and interpreting a research experiment grounded in a peer-reviewed paper. EXP-Bench fills this gap by anchoring each task in a real publication, requiring coordinated success across all four phases.

2.2 Scientific Discovery Benchmarks

Several benchmarks assess AI agents on scientific tasks beyond software engineering. ScienceAgentBench [2] evaluates 102 data-driven discovery tasks with ground truth Python programs. SciCode [15] targets research-level coding from 16 natural science fields. LAB-Bench [6] measures biology research capabilities including literature search and protocol planning. AAAR-1.0 [9] evaluates equation inference, experiment design, and paper weakness detection. BoxingGym [3] assesses experimental design in simulated scientific environments. PaperBench [14] is the closest prior work, evaluating agents on replicating 20 ICML 2024 papers from scratch with rubric-based grading; the best agent achieves 21.0%. EXP-Bench differs by (1) providing agents with the actual code repository (masked, not absent), (2) evaluating a larger and more diverse task set (461 vs. 20), (3) using Docker-based execution validation as a hard, binary signal, and (4) employing conjunctive metrics requiring all four phases to succeed simultaneously, revealing performance collapse not visible in averaged scores.

2.3 AI Research Agents

Beyond benchmarks, several systems propose agents that autonomously conduct research. The AI Scientist [10] generates ideas, runs experiments, and writes papers end-to-end. Agent Laboratory [13] produces code and reports through literature review, experimentation, and writing stages. ResearchAgent [1] leverages knowledge graphs to iteratively generate and refine research ideas. While impressive, these systems are not evaluation benchmarks and lack execution-validated, granular ground truth. EXP-Bench provides the infrastructure needed to rigorously measure how well such systems—or any AI agent—can execute real AI research experiments.

3 The EXP-Bench Benchmark and Dataset

3.1 Task Definition and Structure

Each EXP-Bench task corresponds to one experiment from a peer-reviewed AI paper and is structured to require an agent to replicate that experiment in a realistic research environment. Concretely, an agent receives: (1) a *research question* derived from the paper's experimental claims; (2) a *high-level method description* providing context about the approach; and (3) a *masked code repository* in which answer-revealing files (e.g., README sections with results, run scripts that directly produce paper numbers) have been removed via scripted git operations. The agent must then produce four outputs evaluated by EXP-Bench: a structured *design plan* (D) specifying the experimental variables and procedure; an *implementation* (I) consisting of code changes to the repository that would run the experiment; an *execution* (E) in which the agent's code is actually run in a containerized environment; and a *conclusion* (C) interpreting the results.

Ground truth for each task consists of three components: a design specification written at the subtask level, a git-diff capturing the actual implementation needed to run the experiment, and a validated conclusion drawn from running the unmasked code and comparing with the paper. Tasks are decomposed into an average of 27.6 fine-grained subtasks each, enabling partial-credit scoring within each phase.

Figure 2 illustrates one task drawn from the ICLR 2024 paper MogaNet [7], showing how the research question, repository context, and expected outputs are presented to the agent.

3.2 Data Construction Pipeline

EXP-Bench tasks were constructed through a three-stage semi-automated pipeline, illustrated in Figure 3. The pipeline is designed to produce realistic tasks at scale while maintaining execution-validated quality.

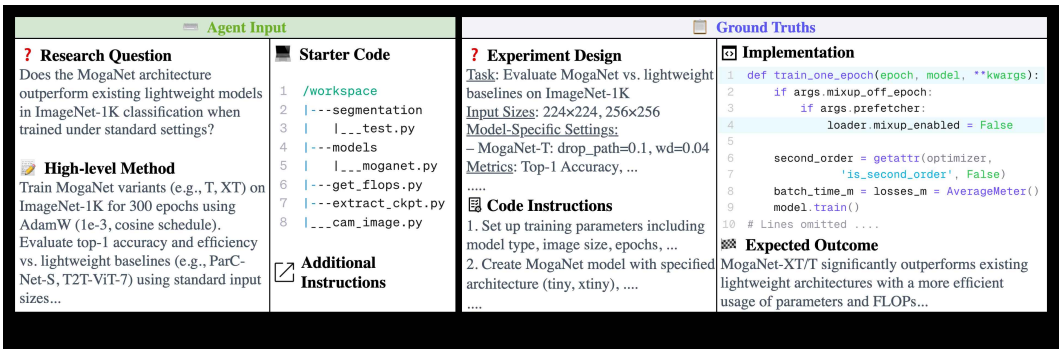

Figure 2: One AI research task example from ICLR 2024 MogaNet [7]. The agent receives a research question, high-level method, and masked repository; it must produce a design, implementation, execution, and conclusion.

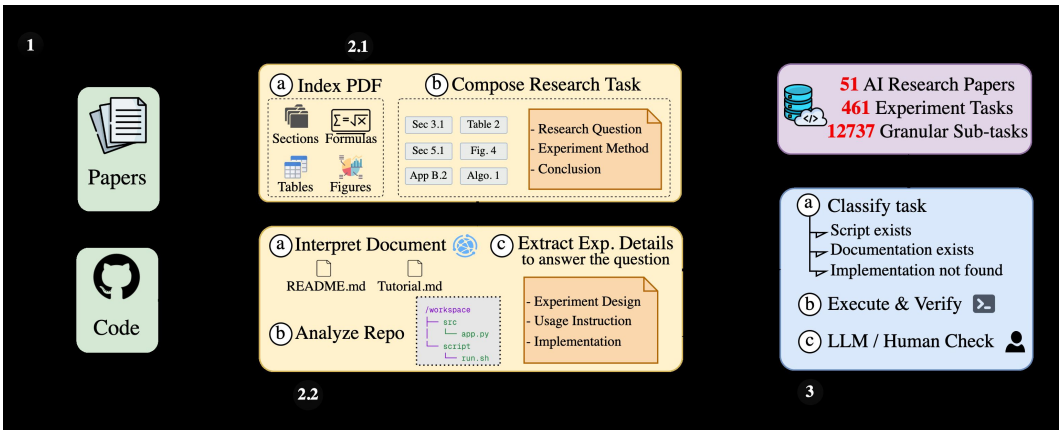

Figure 3: EXP-Bench semi-automated dataset construction pipeline. Tasks are sourced from influential papers (Stage 1), extracted via multimodal parsing and tool-augmented agents (Stage 2), and verified through execution and human review (Stage 3).

Stage 1: Source Selection. We selected papers from NeurIPS and ICLR 2024 with publicly available code repositories. Selection prioritized influential papers using citation count, GitHub stars, and fork activity as signals, ensuring tasks are grounded in well-validated, community-recognized research. This yielded 51 papers spanning computer vision, NLP, reinforcement learning, and applications.

Stage 2: Experiment Procedure Extraction. This stage has two sub-components. *Task extraction* (Stage 2.1) applies OCR and multimodal parsing to paper PDFs, processing figures, tables, and text to identify experimental claims. A retrieval-augmented, multi-pass prompting strategy assembles research questions, methods, datasets, metrics, and experimental constraints. Cross-document re-querying recovers missing details when information is spread across supplementary materials. *Implementation extraction* (Stage 2.2) deploys a tool-augmented agent equipped with PDF, shell, and web access to navigate the repository, locate relevant scripts, identify dependencies and checkpoints, and produce runnable script chains with usage instructions. Abstract syntax tree (AST) tracing converts these scripts into step-by-step implementation requirements that form the subtask decomposition.

Stage 3: Verification and Refinement. Candidate scripts are executed in clean Docker containers, and outputs are compared against paper tables and figures. Failing tasks are iterated on by re-prompting the extraction agents or correcting scripts manually. A lightweight human review pass cross-checks structured task content for factual accuracy and logical consistency. Answer-revealing files—including README sections that describe results, run scripts, and log outputs—are masked

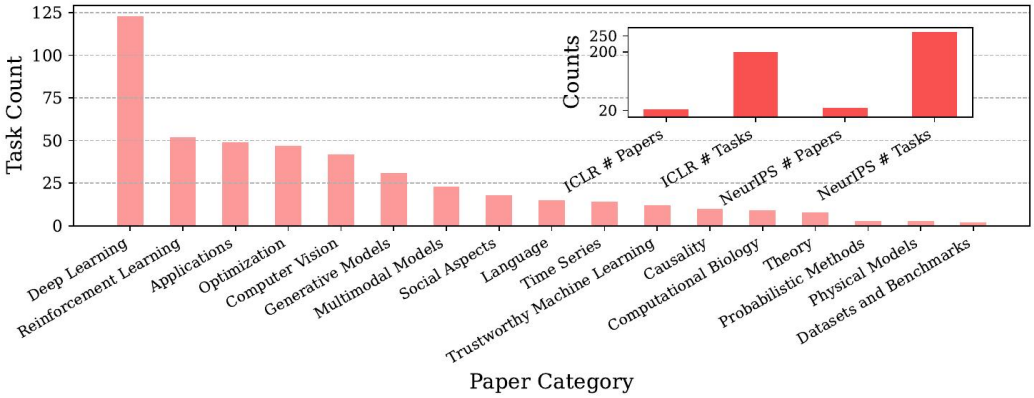

Figure 4: EXP-Bench's dataset comprises tasks from a diverse set of ML research categories, spanning CV, NLP, RL, and application domains from NeurIPS and ICLR 2024 papers.

via scripted git operations. Quality control relies primarily on execution-based validation; no fixed annotator pool or inter-annotator agreement (IAA) metric is reported, which we acknowledge as a limitation of the current pipeline.

3.3 Dataset Statistics and Coverage

The final EXP-Bench dataset comprises 461 tasks from 51 papers, decomposed into 12,737 fine-grained subtasks. Figure 4 shows the distribution of tasks across ML research categories. The dataset spans computer vision (CV), natural language processing (NLP), reinforcement learning (RL), and application domains (e.g., molecular generation, anomaly detection, time-series analysis), reflecting the diversity of experimental work in top-tier AI venues. This breadth ensures that EXP-Bench measures general AI research experimentation capability rather than proficiency in a single subdomain.

4 Evaluation Protocol

4.1 Metrics

EXP-Bench evaluates agents on four dimensions reflecting the phases of a research experiment:

- **Design (D):** Does the agent's experimental plan correctly specify the variables, procedure, and constraints of the experiment? Scored by an o3-mini LLM judge [18] against the ground-truth design specification.
- **Implementation (I):** Does the agent's code correctly implement the experimental procedure? Scored by the LLM judge against the ground-truth git-diff, assessing both structural correctness and coverage of required components.
- **Execution (E):** Does the agent's code run to completion without error? Evaluated as a binary signal by executing the agent's submitted code in a Docker container.
- **Conclusion (C):** Does the agent's stated conclusion correctly interpret the experimental results? Scored by the LLM judge against the validated ground-truth conclusion.

Beyond individual metrics, EXP-Bench introduces *conjunctive* metrics to capture end-to-end success:

- $\mathbf{I} \cdot \mathbf{E}$: Implementation and Execution both pass.
- $\mathbf{All}\checkmark$: Design, Implementation, and Conclusion all pass (D & I & C).
- $\mathbf{All} \cdot \mathbf{E}\checkmark$: All four metrics pass (D & I & E & C).

These conjunctive metrics are essential because an experiment that succeeds at implementation but fails at execution—or draws the wrong conclusion—is not scientifically valid. As our results show,

Table 1: Average benchmark scores for various models when tested against various evaluation metrics. Popular Agents and LLMs perform poorly on EXP-Bench, showcasing its difficulty.

Agent	Model	D	I	E	I·E	C	All✓	All·E✓	#E
OpenHands	o3-mini	18.4	20.3	15.0	2.9	21.0	1.4	0.5	420
OpenHands	Claude-3.7 Sonnet	16.0	35.0	33.2	14.9	13.4	0.7	0.4	235
OpenHands	Amazon Nova Pro	18.2	19.5	26.8	0.0	15.7	0.0	0.0	56
OpenHands	Claude-3.5 Haiku	20.6	26.2	9.3	1.3	13.8	0.0	0.0	237
OpenHands	DeepSeek R1	6.8	10.0	0.7	0.0	2.4	0.0	0.0	140
IterativeAgent	Claude-3.5 Haiku	6.4	20.6	25.2	5.4	2.2	0.0	0.0	111
IterativeAgent	Amazon Nova Pro	0.1	10.0	18.1	0.0	0.3	0.0	0.0	215

conjunctive scoring dramatically reduces apparent agent performance, revealing that individual-metric scores substantially overstate the rate of complete experimental success.

4.2 Execution Environment and Integrity Monitoring

Each agent's submitted code is executed inside a Docker container running Ubuntu 24.04, equipped with 4×A40 GPUs and subject to a 40-minute timeout. The container has access to the (masked) repository, required dependencies, and pretrained checkpoints where applicable. Execution is treated as passing (E=1) only if the code completes without runtime error within the time limit.

An integrity monitor runs alongside each evaluation to detect disallowed behaviors, including: accessing the original paper text or supplementary materials, using git operations to restore masked files, and generating synthetic or fake experimental results. Only evaluation runs that pass the integrity monitor are included in reported scores. This prevents agents from "shortcutting" by reading the answers rather than solving the task.

4.3 Agent and Model Setup

We evaluate two agent frameworks: **OpenHands** [16], an open platform for AI software development agents; and an **IterativeAgent** (IA) framework that iteratively proposes and refines code. These are paired with five LLM backends: o3-mini, Claude-3.7 Sonnet, Claude-3.5 Haiku, DeepSeek R1, and Amazon Nova Pro, yielding seven distinct agent–model configurations. Not all combinations were evaluated on all 461 tasks due to computational constraints; the number of execution-checked tasks ($\#E$) is reported per configuration.

5 Experiments

5.1 Main Results

Table 1 reports average benchmark scores for all agent–model configurations. The results reveal two complementary patterns. First, agents achieve non-trivial scores on individual dimensions: the best Implementation score is **35.0%** (OpenHands+Claude-3.7 Sonnet) and the best Execution rate is **33.2%** (same configuration), suggesting that current agents can make meaningful progress on sub-tasks of AI research experiments. Second, conjunctive all-pass rates collapse dramatically: the best $\mathrm{All} \cdot \mathrm{E}\checkmark$ score across all configurations is only **0.5%** (OpenHands+o3-mini, n=420), and OpenHands+Claude-3.7 Sonnet achieves $\mathrm{All} \cdot \mathrm{E}\checkmark$=0.4% despite its higher individual scores. The average individual metric score across monitor-passing runs is approximately 20.6%, yet the average conjunctive $\mathrm{All} \cdot \mathrm{E}\checkmark$ rate is approximately 0.2%.

OpenHands+DeepSeek R1 performs the worst overall (D=6.8%, I=10.0%, E=0.7%), and IterativeAgent+Amazon Nova Pro achieves near-zero Design scores (D=0.1%), suggesting that model capability is the primary bottleneck for weaker configurations. OpenHands+Claude-3.5 Haiku achieves the highest Design score (20.6%) among the lower-performing configurations, while OpenHands+Amazon Nova Pro achieves 26.8% Execution but 0.0% on all conjunctive metrics, indicating execution without coherent design or conclusion.

Table 2: Average benchmark scores of various models and agents across select task categories; see Supp. **??** for complete list. Evaluation performed against EXP-Bench.

Category	Agent	Model	D	I	E	I·E	C	All✓	All✓·E
Applications	OH	Nova Pro	19.2	23.9	19.0	0.0	13.9	0.0	0.0
Applications	OH	o3-mini	9.0	8.0	0.0	0.0	8.3	0.0	0.0
Applications	OH	3.5 Haiku	19.2	24.5	8.3	5.6	8.3	0.0	0.0
Applications	OH	3.7 Sonnet	9.0	26.8	30.8	7.7	8.3	2.8	0.0
Applications	IA	Nova Pro	0.0	9.8	5.0	0.0	0.0	0.0	0.0
Applications	IA	3.5 Haiku	0.0	18.0	0.0	0.0	0.0	0.0	0.0
Applications	OH	DeepSeek R1	3.1	4.3	0.0	0.0	0.0	0.0	0.0
RL	OH	3.7 Sonnet	18.3	48.2	27.3	21.2	17.6	2.0	3.0
RL	OH	o3-mini	23.5	34.8	15.7	2.0	27.5	3.9	0.0
RL	OH	3.5 Haiku	27.7	41.4	11.5	0.0	17.6	0.0	0.0
RL	IA	3.5 Haiku	3.3	27.5	17.4	0.0	2.4	0.0	0.0
RL	OH	DeepSeek R1	5.0	10.3	0.0	0.0	2.0	0.0	0.0
RL	IA	Nova Pro	0.0	8.6	9.7	0.0	0.0	0.0	0.0
RL	OH	Nova Pro	17.9	28.9	0.0	0.0	13.7	0.0	0.0

Table 3: Agents fail in diverse ways across different phases of experimentation; this table presents a simplified subset of common examples, measured across all agent and model evaluations.

Phase	Failure Type	Prevalence (%)
Design	Incomplete or Misclassified Design Variables	16.05
Design	Irrelevant Procedural Additions in Design	7.62
Implementation	Missing Essential implementation Components	39.71
Implementation	Incomplete Evaluation Metric Implementation	2.15
Implementation	Incomplete Data and Preprocessing Setup	1.83
Execution	Environment/Dependency Configuration Errors	29.38
Execution	Execution Script and File Errors	23.84
Execution	Missing Setup Script File	6.95
Execution	Tensor Operation Execution Error	3.22
Conclusion	Missing Conclusion Content	26.18
Conclusion	Incorrect Conclusion Interpretation	19.66
Conclusion	Extraneous Details in Conclusion	7.77
Conclusion	Incorrect Numeric Conclusion	3.21

5.2 Per-Category Analysis

Table 2 reports results broken down by research category for Applications and Reinforcement Learning tasks. RL tasks are relatively more tractable for agents: OpenHands+Claude-3.7 Sonnet achieves I=**48.2%** and I · E=**21.2%** on RL tasks, substantially above its overall averages. OpenHands+o3-mini reaches All✓=3.9% on RL tasks, the highest task-category all-pass rate observed. Application tasks are generally harder, with most configurations achieving near-zero conjunctive scores.

Figures 5 and 6 illustrate cost–time trade-offs and the performance collapse under stricter conjunctive metrics.

6 Analysis

6.1 Failure Taxonomy

Table 3 presents a taxonomy of agent failures across the four experimental phases, measured across all agent–model evaluations. Failures are diverse and occur at every stage of the pipeline.

The most prevalent failure is *missing essential implementation components* (39.7%), indicating that agents frequently produce incomplete code that omits critical steps such as evaluation loops, data preprocessing, or metric computation. *Environment and dependency configuration errors* (29.4%) and *execution script and file errors* (23.8%) together account for over half of execution failures,

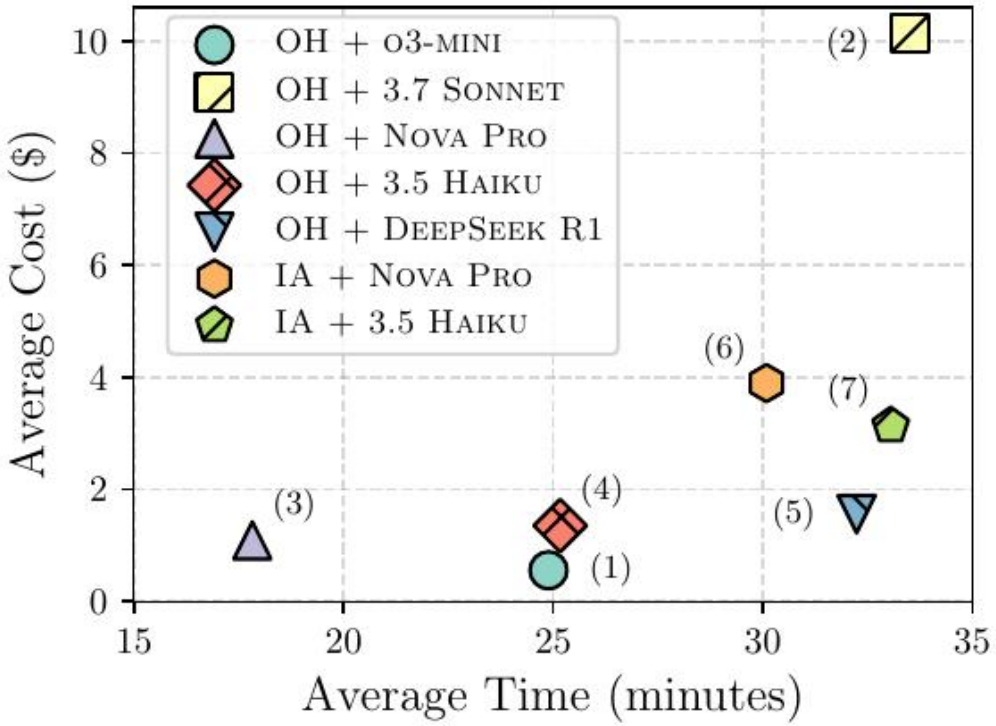

Figure 5: Cost–time trade-offs across agent–model configurations on EXP-Bench. Higher-cost models achieve better individual metric scores but do not proportionally improve conjunctive success rates.

suggesting that dependency management and script organization are significant practical barriers. On the design side, *incomplete or misclassified design variables* (16.1%) reflect difficulties in correctly decomposing an experimental procedure from a high-level description.

Conclusion failures are also common: 26.2% of runs produce missing conclusion content, and 19.7% produce incorrect interpretations. This is notable because the conclusion phase is evaluated *after* execution, meaning that agents often fail to correctly interpret results even when they partially succeed at running code. These patterns collectively suggest that agents lack the integrated understanding needed to coordinate design, implementation, execution, and interpretation as a coherent whole.

6.2 Benchmark Reliability and Stability

Figure 7 presents a stability analysis of EXP-Bench scores, examining variability across repeated evaluations of the same agent–model configuration. Score variance is low, confirming that the LLM judge and execution validator produce consistent signals across runs.

The use of o3-mini as the LLM judge introduces a potential source of bias, as judge assessments may not perfectly align with expert human judgment. We partially mitigate this by using structured rubrics derived from ground-truth subtask decompositions rather than holistic judgment. The lightweight human review in the construction pipeline provides a further quality check on ground-truth correctness, but we acknowledge that a formal inter-annotator agreement study would strengthen the benchmark's validity.

7 Discussion

The central finding of EXP-Bench—that individual-phase success does not imply end-to-end success—has practical implications for developing AI research agents. Systems that perform well on isolated

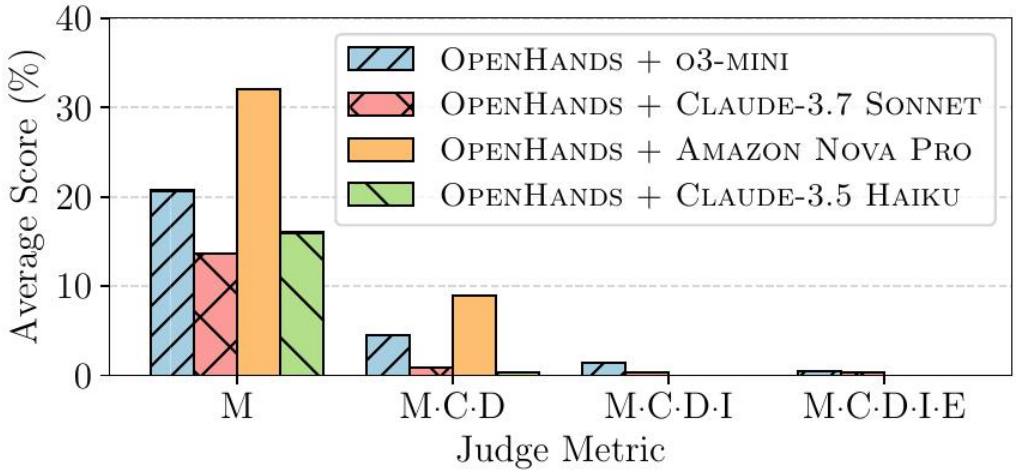

Figure 6: Stricter conjunctive metrics reveal substantially lower true end-to-end correctness. Average scores drop steeply when requiring all four phases (D, I, E, C) to pass simultaneously.

benchmarks may still fail to complete real experiments, because failures often arise in cross-phase coordination: a design that does not match the implementation, or an implementation that executes but produces results the agent cannot interpret. The conjunctive metric framework is designed to expose exactly this kind of failure.

The stronger performance on RL tasks (e.g., I=48.2% for OpenHands+Claude-3.7 Sonnet) suggests that domain characteristics matter: RL experiments may be more self-contained with clearer codebases. Future work could investigate whether EXP-Bench performance predicts success on real research workflows.

Limitations. EXP-Bench has several important limitations. First, tasks are drawn exclusively from NeurIPS and ICLR 2024, which may not generalize to other venues, older papers, or non-ML scientific domains. Second, the quality control pipeline relies on execution-based validation and lightweight human review rather than formal inter-annotator agreement, which may allow some annotation errors to persist. Third, the LLM judge (o3-mini) may have systematic biases in scoring Design and Conclusion, particularly for tasks requiring domain-specific knowledge. Fourth, the 40-minute execution timeout may disadvantage legitimate tasks that require longer training runs, potentially underestimating agent execution success for such tasks.

8 Conclusion

We presented EXP-Bench, the first benchmark to evaluate AI agents on complete, executable AI research experiments grounded in peer-reviewed publications. EXP-Bench comprises 461 tasks from 51 NeurIPS/ICLR 2024 papers with 12,737 subtasks spanning CV, NLP, RL, and applications, evaluated via a conjunctive protocol combining LLM judging, Docker-based execution, and integrity monitoring. Across seven agent–model configurations, individual-phase scores reach up to 35% (implementation) and 33% (execution), but end-to-end all-pass rates remain below 1%, with missing implementation components, environment errors, and incorrect conclusions as dominant failure modes. EXP-Bench exposes the gap between partial and complete AI research experimentation and provides a challenging frontier for autonomous AI research agents.

References

[1] J. Baek, S. K. Jauhar, S. Cucerzan, and S. J. Hwang. Researchagent: Iterative research idea generation over scientific literature with large language models. *arXiv preprint arXiv:2404.07738*, 2024.

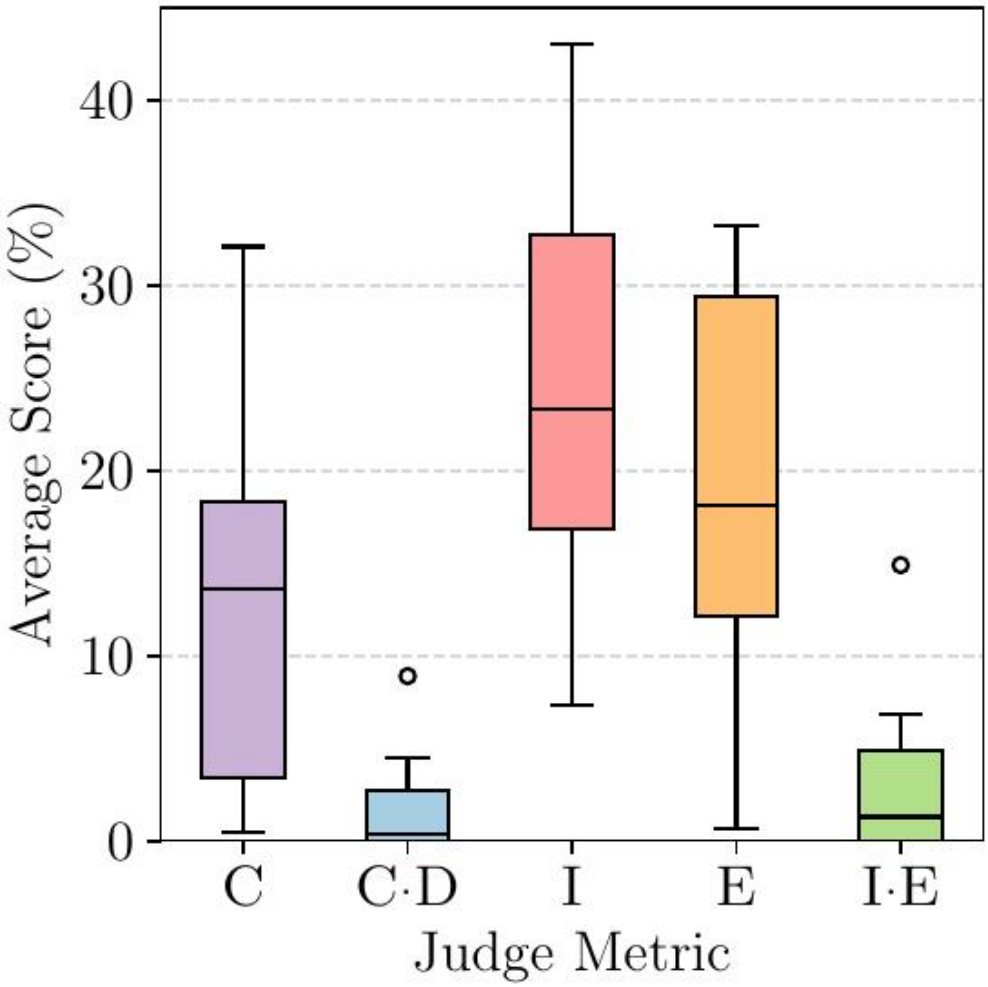

Figure 7: Stability analysis of EXP-Bench. Repeated evaluations of the same configuration show low score variance, confirming that the LLM judge and execution validator provide consistent, reproducible signals.

[2] Z. Chen, S. Chen, Y. Ning, Q. Zhang, B. Wang, B. Yu, Y. Li, Z. Liao, C. Wei, Z. Lu, et al. Scienceagentbench: Toward rigorous assessment of language agents for data-driven scientific discovery. *arXiv preprint arXiv:2410.05080*, 2024.

[3] K. Gandhi, M. Y. Li, L. Goodyear, L. Li, A. Bhaskar, M. Zaman, and N. D. Goodman. Boxing-gym: Benchmarking progress in automated experimental design and model discovery, 2025.

[4] Q. Huang, J. Vora, P. Liang, and J. Leskovec. Mlagentbench: Evaluating language agents on machine learning experimentation, 2024.

[5] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

[6] J. M. Laurent, J. D. Janizek, N. Thakkar, M. Ruzo, M. S. Yao, M. S. Levine, S. G. Rodriques, and A. White. Lab-bench: Measuring capabilities of language models for biology research, 2024.

[7] S. Li, Z. Wang, Z. Liu, C. Tan, H. Lin, D. Wu, Z. Chen, J. Zheng, and S. Z. Li. Moganet: Multi-order gated aggregation network, 2024.

[8] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang. Agentbench: Evaluating llms as agents, 2023.

[9] R. Lou, H. Xu, S. Wang, J. Du, R. Kamoi, X. Lu, J. Xie, Y. Sun, Y. Zhang, J. J. Ahn, H. Fang, Z. Zou, W. Ma, X. Li, K. Zhang, C. Xia, L. Huang, and W. Yin. Aaar-1.0: Assessing ai's potential to assist research, 2024.

[10] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024.

[11] D. Nathani, L. Madaan, N. Roberts, N. Bashlykov, A. Menon, V. Moens, A. Budhiraja, D. Magka, V. Vorotilov, G. Chaurasia, D. Hupkes, R. S. Cabral, T. Shavrina, J. Foerster, Y. Bachrach, W. Y. Wang, and R. Raileanu. Mlgym: A new framework and benchmark for advancing ai research agents, 2025.

[12] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez. Gorilla: Large language model connected with massive apis, 2023.

[13] S. Schmidgall, Y. Su, Z. Wang, X. Sun, J. Wu, X. Yu, J. Liu, Z. Liu, and E. Barsoum. Agent laboratory: Using llm agents as research assistants. *arXiv preprint arXiv:2501.04227*, 2025.

[14] G. Starace, O. Jaffe, D. Sherburn, J. Aung, J. S. Chan, L. Maksin, R. Dias, E. Mays, B. Kinsella, W. Thompson, et al. Paperbench: Evaluating ai's ability to replicate ai research. *arXiv preprint arXiv:2504.01848*, 2025.

[15] M. Tian, L. Gao, S. D. Zhang, X. Chen, C. Fan, X. Guo, R. Haas, P. Ji, K. Krongchon, Y. Li, S. Liu, D. Luo, Y. Ma, H. Tong, K. Trinh, C. Tian, Z. Wang, B. Wu, Y. Xiong, S. Yin, M. Zhu, K. Lieret, Y. Lu, G. Liu, Y. Du, T. Tao, O. Press, J. Callan, E. Huerta, and H. Peng. Scicode: A research coding benchmark curated by scientists, 2024.

[16] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song, B. Li, J. Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.

[17] H. Wijk, T. Lin, J. Becker, S. Jawhar, N. Parikh, T. Broadley, L. Chan, M. Chen, J. Clymer, J. Dhyani, E. Ericheva, K. Garcia, B. Goodrich, N. Jurkovic, M. Kinniment, A. Lajko, S. Nix, L. Sato, W. Saunders, M. Taran, B. West, and E. Barnes. Re-bench: Evaluating frontier ai rd capabilities of language model agents against human experts, 2024.

[18] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.